

Laporan Praktikum Minggu Ke-5 Kontrol Cerdas

Minggu Ke-5

Nama : Mohammad Arief Rachman
NIM : 224308060
Kelas : TKA 7C
Akun Github (Tautan) : <https://github.com/moharief0304>

1. Judul Laporan

Deep reinforcement Learning untuk kontrol kompleks

2. Tujuan Percobaan

- Memahami konsep Deep Reinforcement Learning (DRL) dalam kontrol sistem kompleks.
- Mengimplementasikan Deep Q-Network (DQN) untuk kontrol otomatis.
- Menganalisis performa DRL dibandingkan dengan metode kontrol konvensional.

3. Landasan Teori

Deep reinforcement learning (deep RL) adalah gabungan dari dua pendekatan utama dalam kecerdasan buatan, yaitu *reinforcement learning* (pembelajaran penguatan) dan *deep learning* (pembelajaran mendalam). Dua pendekatan ini memiliki kekuatan masing-masing, dan ketika digabungkan, mampu menciptakan sistem AI yang bisa belajar dan beradaptasi dalam lingkungan yang sangat kompleks. *Reinforcement learning* adalah metode pembelajaran di mana sistem AI belajar mengambil keputusan melalui trial and error, atau mencoba berbagai tindakan dalam suatu lingkungan dan menerima umpan balik berupa imbalan (reward) atau hukuman (penalti). Misalnya, dalam permainan catur, sebuah AI akan belajar bahwa langkah-langkah tertentu bisa membawa kemenangan (reward), sementara langkah lainnya bisa menyebabkan kekalahan (penalty).

Sementara itu, *deep learning* adalah metode yang menggunakan *artificial neural networks* dengan banyak lapisan (deep) untuk menganalisis data dan mengenali pola yang sangat kompleks. Teknologi ini telah membawa terobosan besar dalam pengenalan wajah, pengolahan bahasa alami, dan visi komputer.

Deep Q Network (DQN) adalah perluasan dari *Q-Learning* yang menggunakan jaringan saraf tiruan untuk memperkirakan nilai-Q, alih-alih mengandalkan Tabel-Q yang besar. Dalam Q-Learning tradisional, Tabel-Q menyimpan nilai untuk setiap pasangan keadaan-aksi, yang dengan cepat menjadi tidak praktis untuk lingkungan dengan ruang keadaan yang besar atau kontinu. DQN mengatasi hal ini dengan menggunakan jaringan saraf tiruan untuk mengaproksimasi nilai-Q ini. Dalam kerangka kerja DQN, ada anggapan tersembunyi bahwa keadaan yang kita definisikan adalah pengamatan penuh terhadap lingkungan pada setiap langkah waktu. Namun, untuk lingkungan perdagangan valas, variabel yang ada dalam dimensi waktu biasanya tidak dapat ditemukan dengan baik oleh kerangka kerja DQN.

4. Analisis dan Diskusi

Pada Deep Q-Network (DQN) punya beberapa keunggulan dibanding metode Q-Learning biasa, khususnya dalam menangani sistem rumit dan ruang state yang luas. Di Q-Learning tradisional, proses belajarnya pakai Q-Table yang menyimpan nilai untuk setiap kombinasi state dan action. Cara ini bagus untuk sistem sederhana dengan jumlah state yang terbatas, tapi jadi kurang praktis kalau ruang state-nya sangat besar atau berbentuk kontinu. DQN mengatasi masalah ini dengan memakai neural network untuk memperkirakan nilai Q secara langsung, jadi bisa bekerja di lingkungan yang lebih kompleks dan dinamis.

Selain itu, DQN juga dilengkapi dengan mekanisme experience replay yang memungkinkan pengambilan sampel pengalaman secara acak saat training. Teknik ini berguna untuk mengurangi korelasi antar data dan membuat proses pembelajaran lebih stabil. Strategi epsilon-greedy yang digunakan DQN juga membantu menjaga keseimbangan antara exploration dan exploitation, sehingga agent bisa belajar lebih efektif dari lingkungannya.

Namun di sisi lain, DQN juga punya beberapa kekurangan. Algoritma ini butuh resource komputasi yang cukup besar karena melibatkan proses training neural network yang kompleks. Waktu training-nya juga relatif lebih lama dibanding Q-Learning klasik. Selain itu, performa DQN sangat tergantung pada pengaturan parameter seperti learning rate, discount factor, dan ukuran network. Kalau parameter-nya tidak disetel dengan benar, model bisa mengalami overfitting atau pembelajaran yang tidak stabil. Jadi meskipun DQN menawarkan kemampuan yang lebih advanced, implementasinya butuh keahlian teknis dan perencanaan yang matang.

Untuk menerapkan DQN pada kontrol sistem dinamis seperti drone atau robot industri, ada beberapa penyesuaian yang perlu dilakukan supaya algoritma bisa bekerja dengan baik di lingkungan yang terus berubah-ubah. Langkah awal yang penting adalah menentukan state dan action dengan tepat. Pada sistem drone misalnya, state bisa mencakup posisi, orientasi, dan kecepatan,

sedangkan action-nya bisa berupa perubahan sudut propeller atau thrust. Data dari sensor seperti IMU, kamera, atau encoder dipakai sebagai input supaya agent DQN bisa mendapat informasi akurat tentang kondisi sistem secara real time.

Training DQN untuk sistem dinamis biasanya dimulai di lingkungan simulasi seperti OpenAI Gym, Gazebo, atau PyBullet. Simulasi digunakan untuk menghindari risiko kerusakan perangkat fisik selama proses pembelajaran awal. Setelah model menunjukkan hasil yang stabil di simulasi, baru bisa diterapkan ke perangkat nyata melalui proses transfer learning. Selain itu, dalam implementasi di dunia nyata, DQN sering digabungkan dengan metode kontrol konvensional seperti PID atau Fuzzy Logic untuk menjaga stabilitas sistem saat model masih dalam tahap adaptasi.

Supaya sistem bisa belajar dengan optimal, reward function harus dirancang secara hati-hati sesuai dengan tujuan kontrol, misalnya untuk menjaga keseimbangan drone atau mengarahkan robot ke posisi target secara efisien. Dengan desain yang tepat, DQN bisa beradaptasi secara mandiri terhadap perubahan lingkungan dan menghasilkan kontrol yang presisi pada sistem dinamis yang kompleks.

5. Assignment

Dalam tugas minggu kelima ini dikembangkan sistem pengenalan gestur tangan real-time menggunakan Support Vector Machine (SVM) dan MediaPipe yang terintegrasi dengan OpenCV. Proses dimulai dengan memuat dataset keypoint tangan menggunakan `pd.read_csv()`, kemudian menyesuaikan nama kolom agar seragam. Dataset diasumsikan memiliki label pada kolom pertama dan noise pada kolom terakhir, sehingga fitur (X) diambil dari kolom ke-2 hingga sebelum kolom terakhir, sedangkan label (y) berasal dari kolom pertama. Data dibagi menjadi 80% untuk pelatihan dan 20% untuk pengujian menggunakan `train_test_split()`.

Model SVM dengan kernel linear dilatih menggunakan `SVC(kernel='linear')` dan diuji pada data uji untuk menghitung tingkat akurasi melalui `accuracy_score()`. Setelah pelatihan, model diterapkan dalam deteksi gestur tangan secara real-time dengan MediaPipe dan OpenCV. Kamera menangkap citra, mengonversinya ke format RGB, lalu mendeteksi landmark tangan menggunakan `hands.process()`. Jika tangan terdeteksi, koordinat x, y, dan z setiap titik diekstraksi dan dimasukkan ke model SVM

untuk memprediksi gestur. Hasil prediksi ditampilkan di layar menggunakan cv2.putText().

Dengan menggabungkan MediaPipe untuk pelacakan titik kunci tangan dan SVM untuk klasifikasi gestur, sistem ini mampu mengenali posisi serta gerakan tangan secara akurat dalam waktu nyata, menunjukkan penerapan efektif antara *machine learning* dan *computer vision* dalam analisis gerakan manusia. Berikut ini adalah fungsi dari kode program yang digunakan seperti pada tabel berikut ini :

No	Bagian Kode / Fungsi	Penjelasan
1	<code>import cv2</code>	Mengimpor pustaka OpenCV untuk menangani pengolahan citra dan akses kamera.
2	<code>import mediapipe as mp</code>	Mengimpor pustaka MediaPipe untuk melakukan deteksi dan pelacakan tangan secara real-time.
3	<code>mp_hands = mp.solutions.hands</code>	Menginisialisasi modul Hands dari MediaPipe yang berfungsi mendeteksi dan melacak titik tangan (landmarks).
4	<code>mp_drawing = mp.solutions.drawing_utils</code>	Menginisialisasi fungsi utilitas untuk menggambar hasil deteksi landmark di atas frame video.
5	<code>cap = cv2.VideoCapture(0)</code>	Membuka koneksi ke kamera (0 berarti kamera bawaan laptop). Jika ada kamera eksternal, indeks bisa diganti (misal 1 atau 2).
6	<code>with mp_hands.Hands(...) as hands:</code>	Membuat objek deteksi tangan dengan parameter: - max_num_hands=2: mendeteksi maksimal dua tangan. - min_detection_confidence=0.7 : batas minimal kepercayaan

		deteksi. - min_tracking_confidence=0.7: batas minimal kepercayaan pelacakan.
7	<code>ret, frame = cap.read()</code>	Membaca satu frame dari kamera. <code>ret</code> bernilai <i>True</i> jika pembacaan berhasil.
8	<code>cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)</code>	Mengubah format warna dari BGR (OpenCV) ke RGB (MediaPipe membutuhkan format RGB).
9	<code>results = hands.process(rgb_frame)</code>	Memproses frame RGB untuk mendeteksi tangan dan menghasilkan koordinat titik-titik (landmarks).
10	<code>if results.multi_hand_landmarks:</code>	Mengecek apakah ada tangan yang terdeteksi pada frame.
11	<code>for hand_landmarks in results.multi_hand_landmarks:</code>	Melakukan iterasi untuk setiap tangan yang terdeteksi (bisa satu atau dua).
12	<code>mp_drawing.draw_landmarks(...)</code>	Menggambar titik-titik sendi dan koneksi antar sendi tangan di atas frame video dengan warna dan ketebalan tertentu.
13	<code>cv2.imshow("Deteksi Gerakan Tangan - Titik Sendi", frame)</code>	Menampilkan hasil deteksi tangan secara real-time di jendela OpenCV.
14	<code>if cv2.waitKey(1) & 0xFF == ord('q'):</code>	Memberi opsi keluar dari program jika tombol 'q' ditekan di keyboard.
15	<code>cap.release()</code>	Menutup koneksi kamera setelah program berhenti.
16	<code>cv2.destroyAllWindows()</code>	Menutup semua jendela tampilan OpenCV.

6. Data dan Output Hasil Pengamatan

No	Dokumentasi	Deteksi	Jumlah Titik Sendi per Tangan	Posisi/Titik Penting	Keterangan Output Visual
1		Gerakan Tangan terbuka lebar Jumlah Tangan Terdeteksi 2	21 titik per tangan	Semua jari terbuka, jari-jari terlihat jelas	Garis hijau menghubungkan sendi, titik merah di setiap sendi
2		Gerakan Tangan membentuk "jempol dan telunjuk" Jumlah Tangan Terdeteksi 1	22 titik	Jari telunjuk & jari jempol terangkat, jari lain menekuk	Garis hijau mengikuti bentuk jari, visualisasi sendi tetap akurat
3		Gerakan Tangan terbuka normal Jumlah Tangan Terdeteksi 1	21 titik	Semua jari terbuka, posisi natural	Deteksi sendi stabil, tidak ada tangan kedua

7. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan dapat disimpulkan bahwa:

- 1) Program berhasil melakukan klasifikasi gestur tangan secara real-time dengan menggabungkan model SVM dan MediaPipe
- 2) Akurasi model sangat dipengaruhi oleh kualitas dataset dan parameter yang digunakan dalam training
- 3) Sistem mampu mengenali gerakan tangan dengan hasil yang cukup baik dalam kondisi normal
- 4) Sistem memiliki keterbatasan yaitu hanya bisa mendeteksi maksimal dua tangan dalam satu waktu

8. Saran

Berdasarkan hasil praktikum dan analisis yang telah dilakukan, ada beberapa saran perbaikan yang bisa diterapkan untuk meningkatkan akurasi dan performa sistem. Pertama, perlu dilakukan peningkatan dataset dengan menambah jumlah dan variasi sampel supaya mencakup berbagai kondisi pencahayaan dan posisi tangan yang berbeda-beda, sehingga model bisa lebih general dan akurat. Kedua, optimasi model dan performa real-time dapat dilakukan melalui hyperparameter tuning pada SVM seperti parameter C dan gamma, optimasi kode program, kompresi gambar, serta menambahkan post-processing seperti time smoothing agar hasil prediksi lebih stabil. Ketiga, kapasitas deteksi sistem perlu ditingkatkan supaya bisa mendeteksi lebih dari dua tangan atau bahkan objek-objek relevan lainnya dalam satu waktu. Dengan implementasi perbaikan-perbaikan ini, diharapkan sistem dapat bekerja lebih optimal dan robust dalam berbagai kondisi penggunaan.

9. Daftar Pustaka

Apa Itu Pembelajaran Penguatan Mendalam? Teknologi di Balik AI

Pintar . (nd). Pusat Kecerdasan Buatan

Indonesia. <https://aihub.id/pengetahuan-dasar/deep-reinforcement-learning>

Z. Jiang D. Xu and J. Liang “A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem ” arXiv [q-fin.CP], 2017

Ramesh, S., N, S. B., Sathyavarapu, S. J., Sharma, V., A, N. K. A., & Khanna, M. (2025). Comparative analysis of Q-learning, SARSA, and deep Q-network for microgrid energy management. *Scientific Reports*, 15(1). <https://doi.org/10.1038/s41598-024-83625-8>